Improved DDPG Based Two-Timescale Multi-Dimensional Resource Allocation for Multi-Access Edge Computing Networks

Qianqian Liu, Haixia Zhang, *Senior Member, IEEE*, Xin Zhang, and Dongfeng Yuan, *Senior Member, IEEE*

***Abstract*—Due to the dependence of task processing on the task-related models and databases, edge service caching and multi-access edge computing (MEC) are tightly coupled. The service caching placement, offloading and resource allocation have become the key to guarantee the latency and computing requirements of tasks. However, much of the existing work is based on unrealistic assumptions that caching and other resources can be scheduled simultaneously, which lead to frequent cache switching and high system overhead. To address this, we propose a two-timescale multi-dimensional resource optimization scheme based on a Markov decision process (MDP), which jointly optimizes the long-term service caching and short-term task offloading, computing, and bandwidth resource allocation to minimize the long-term system delay and cache cost. To accommodate the two-timescale characteristics, we propose a centralized dual-actor deep deterministic policy gradient (DDPG) algorithm, where there is a dual-actor network producing both long-term and short-term resource management decisions, and a centralized critic network directing joint actions. Such a structure strengthens the learning process, enabling the dual-actor network to generate different timescale decisions that align with consistent objectives. Simulation results show that the proposed algorithm can reduce delay and cache cost compared to the existing scheme.**

***Index Terms*—Service caching, multi-dimensional resources allocation, multi-access edge computing, two-timescale, centralized dual-actor deep deterministic policy gradient.**

I. Introduction

Multi-access Edge Computing (MEC) has been proposed as a key technology, which complements the limitations of cloud computing by bringing computing and storage resources closer to the edge [1]. Thus, resource-restricted terminals can meet the latency and computing requirements of tasks by offloading them to the edge. Considering the resource limitations of the edge server (ES), appropriate offloading and resource allocation strategies have been widely studied [2], [3]. Meanwhile, offloaded tasks, particularly those driven by artificial intelligence, can only be directly processed by the ES if the corresponding models and databases have been cached in advance. This is known as edge service caching [4], [5], [6].

Since the service caching and offloading are tightly coupled with each other, works have been done to jointly designing the service caching placement, offloading and computing resource allocation to reduce the task completion delay and energy consumption [7], [8], [9], [10]. These works assume that service caching and computing can be done in the same timescale. This is difficult to be implemented in actual systems, where the service caching decisions are typically made at the timescale of hours or days, while the computing decisions are made at a level of seconds or minutes. Without considering the timescale mismatch and doing them in the same timescale will lead to a huge overhead caused by frequently updating service caching, or a large amount of tasks accumulation at the CS, making it difficult to achieve long-term optimal system performance.

To reduce the caching overhead, Zhou et al. [11] has taken into account the dual-timescale nature of service caching and offloading operations, and has made caching placement decision every T slots, and adjust UAV trajectory, user association, and task offloading decisions at each time slot t within T. Wen et al. [12] proposed a two-timescale joint policy for the caching, offloading, and time allocation to minimize the weighted sum energy consumption subject to the storage and delay constraints. However, the large timescale caching strategy is done based solely on the static statistics. To capture time-varying cache requirements caused by offloading decisions, Zhang et al. [13] proposed a hierarchical deep deterministic policy gradient (DDPG)-based collaborative service caching and computing strategy to minimize the costs of latency and energy consumption, but the cache cost and communication resource allocation are not considered in this work. In fact, communication and service caching are prerequisites for computing, and multi-dimensional resources (communication, computing and service caching) jointly determine the end-to-end latency of task processing and system cache cost. These have motivated us to investigate the multi-dimensional resources within two timescales jointly to minimize the long-term system delay and cache cost.

To address this problem, we formulate an optimization problem to jointly determine the long-term service caching, the short-term offloading, as well as the computing and bandwidth resource allocation. Due to the strong coupling and cross-timescale characteristics of the involved variables, it is difficult to find an optimal solution for the optimization problem. To solve it, we transform the formulated optimization problem into a Markov decision process (MDP). Traditionally, deep deterministic policy gradient (DDPG) can be adopted to solve the MDP. But, considering an agent only has one actor network, DDPG cannot output different-dimensional decisions at different timescales. In addition, if adopting two separate agents for each timescale, the critic network of each agent independently guides their decisions, there will be no centralized guidance between actions across timescales [13]. This may result in local optimum or even ineffective decisions. To adopt DDPG to solve the transformed problem, we improve the network structure of DDPG and propose a Centralized Dual-actor DDPG algorithm, where a dual-actor network guided by a centralized critic network is proposed to generate the long-term service caching decisions and the short-term task offloading, computing, and bandwidth resource allocation. Such a structure not only has the advantage of centralized training (a global connection between all decisions), but also can flexibly configure decisions on different timescales.

II. System Model and Problem Formulation

A. System Architecture

We consider a general three-tier MEC network as shown in Fig. 1, which consists of a cloud server (CS), an edge server (ES), and $N =$

Manuscript received 16 August 2023; revised 23 November 2023 and 24 January 2024; accepted 26 January 2024. Date of publication 1 February 2024; date of current version 20 June 2024. This work was supported in part by the Joint Funds of the NSFC under Grant U22A2003 and Grant U23A20277. The review of this article was coordinated by Dr. Antonella Molinaro. *(Corresponding author: Haixia Zhang.)*

Qianqian Liu, Haixia Zhang, and Xin Zhang are with the Shandong Key Laboratory of Wireless Communication Technologies, Shandong University, Jinan 250061, China, and also with the School of Control Science and Engineering, Shandong University, Jinan 250061, China (e-mail: liuqiansdu@mail.sdu.edu.cn; haixia.zhang@sdu.edu.cn; xin.zhang0892@mail.sdu.edu.cn).

Dongfeng Yuan is with the Shandong Key Laboratory of Wireless Communication Technologies, Shandong University, Jinan 250061, China, and also with the School of Qilu Transportation, Shandong University, Jinan 250002, China (e-mail: dfyuan@sdu.edu.cn).

Digital Object Identifier 10.1109/TVT.2024.3360943

0018-9545 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:53:18 UTC from IEEE Xplore. Restrictions apply.

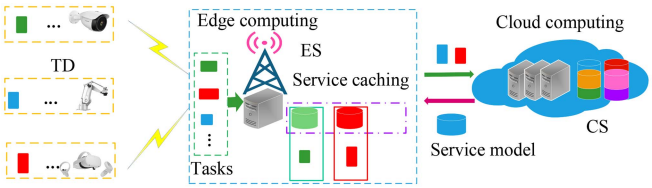


Fig. 1. Process of communication, computing and service caching in a three-tier MEC network. All the tasks will be offloaded to the ES once they are generated. If the ES has cached the dedicated service model of a task, the task will be handled only by the ES, when its computing resource is sufficient (*task green*), or the task will be partially offload to the CS for processing (*task red*), when the computing resource of the ES is not enough. Otherwise, if the needed service model is not cached at the ES, the task will be offloaded to the CS for processing (*task blue*). During the above process, if *task blue* is frequently requested, the ES will download the related service model from the CS.

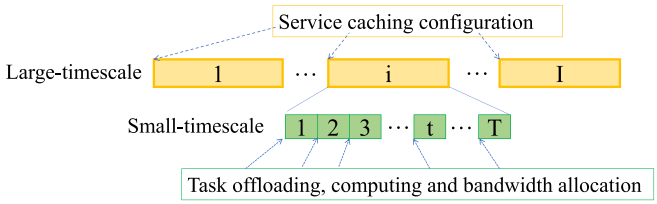


Fig. 2. Two-timescale discrete time model.

$\{1, 2, \ldots, N\}$ randomly distributed terminal devices (TDs), which have no computing capability and can only offload their tasks to the ES and/or the CS for processing. The ES is an integrated MEC server access point (AP), which is responsible for configuring the service caching and scheduling the tasks of all TDs. We adopt a discrete time model which is divided into two-timescale, i.e., large-timescale for service caching placement and small-timescale for task offloading and resource allocation [14]. The large-timescale index of caching is denoted by $i \in \mathcal{I} = \{1, 2, \ldots, I\}$, each of which includes T small-timescale slots. The small-timescale index of the ith large-timescale slot is represented by $t \in \mathcal{T} = \{1, 2, \ldots, T\}$ as shown in Fig. 2. At each small-timescale slot t, each TD randomly generates an arbitrarily divisible task. We use a three-tuple $\{s^{type}_{n,i,t}, s^{input}_{n,i,t}, s^{comp}_{n,i,t}\}$ to denote the task of the nth TD, where $s^{type}_{n,i,t}$ represents the type of the task, $s^{input}_{n,i,t}$ represents the amount of input data (in bits), $s^{comp}_{n,i,t}$ represents the required computational capacity (in CPU cycles).

B. Service Caching Model

There are $\mathcal{F} = \{1, \ldots, f, \ldots, F\}$ kinds of services in the system, and each requested task is related to a specific service model. The ES updates its service caching decisions $\mathbf{c}(i) = \{c^i_1, \ldots, c^i_f, \ldots, c^i_F\}, c^i_f \in 0, 1$ at the beginning of each large-timescale i and maintains the cache configuration unchanged during the inner small-timescale slots. If the ES cached the fth service model at the ith large-timescale slot, $c^i_f = 1$, otherwise, $c^i_f = 0$. Services in $\mathcal{F}$ are all available at the CS and can be cached to the ES via a backhaul, the cost is defined as the communication delay cost for downloading the services,

$$Cost(i) = \eta \sum_{f \in F} \mathbb{I}[(c^i_f - c^{i-1}_f) = 1] \frac{s^{cache}_f}{R^{back}}, \tag{1}$$

where η is a balancing factor, and $\mathbb{I}[x]$ is an indicator function that equals 1 if condition x is true and 0 otherwise, $(c^i_f - c^{i-1}_f) = 1$ indicates that the service should be obtained from the CS, s^{cache}_f is the storage size of the fth service model (in bits), R^{back} is the fixed backhaul transmission rate. Obviously, the cost is 0 at small-timescale slots within a caching period.

C. Communication Model

Considering that the data size of task processing results is negligible compared to the data size of the tasks themselves, in this work, we only focus on the uplink transmission of task offloading when modeling the transmission delay. We assume that TDs and the ES are connected through wireless channels. We define $\mathbf{b}(t) = \{b^i_{1,t}, \ldots, b^i_{n,t}, \ldots, b^i_{N,t}\}$, $b^i_{n,t} \in [0, 1]$ to denote the bandwidth allocation scheme, where $b^i_{n,t}W$ represents the bandwidth allocated to the nth TD by the ES for task offloading. So the offloading rate from TD n to the ES can be written as

$$R^{up}_{n,i,t} = b^i_{n,t} W log_2 \left(1 + \frac{p_n h_{n,i,t}}{\sigma^2}\right), \tag{2}$$

where p_n is the transmit power of the nth TD, $h_{n,i,t} = g_{n,i,t} dis_n^{-\beta}$ represents the corresponding channel gain, with $g_{n,i,t}$ and $dis_n^{-\beta}$ are the small-scale Rayleigh fading and the large-scale path fading, respectively, and dis_n is the distance between TD n and the ES, β is path loss exponent, σ^2 denotes the variance of the additive white Gaussian noise. We assume that there is a wired connection between the ES and the CS, and the transmission rate is defined to be R^c (in bits per second, bps). Without loss of generality, we assume the rate R^c is fixed. So, the transmission delay from the TD n to the CS can be defined as $\frac{s^{input}_{n,i,t}}{R^{up}_{n,i,t}} + \frac{s^{input}_{n,i,t}}{R^c}$.

D. Computing Model

We assume that the tasks are arbitrarily divisible. The ES updates the offloading strategy $\mathbf{d}(t) = \{d^i_{1,t}, \cdots, d^i_{n,t}, \cdots d^i_{N,t}\}$, $d^i_{n,t} \in [0, 1]$ at the beginning of each small-timescale slot t, where $d^i_{n,t} s^{comp}_{n,i,t}$ denotes the portion of the task being processed at the ES, and $(1 - d^i_{n,t}) s^{comp}_{n,i,t}$ represents the remaining portion of the task to be processed by the CS at the tth small-timescale slot within the ith large-timescale slot. Besides, defining $\mathbf{f}(t) = \{f^i_{1,t}, \cdots, f^i_{n,t}, \cdots f^i_{N,t}\}$, $f^i_{n,t} \in [0, 1]$ as the computing resource allocation strategy, $f^i_{n,t} F^e$ represents the computing capacity allocated to TD n by the ES at the tth small-timesacle slot within the ith large-timesacle slot, It should be noted that the task can only be processed at the ES if and only if the relevant service model is cached at the ES and the necessary computing resources are allocated. Hence, we have

$$d^i_{n,t} \leq \lceil f^i_{n,t} \rceil \leq c^i_f, n \in \mathcal{N}, \tag{3}$$

where the ceiling of the value of $f^i_{n,t}$ is used to indicate whether computing resource is allocated to TD n. Therefore, when the ES caches the relevant service model, the delay for processing the task of TD n can be represented as

$$T^{edge}_{n,i,t} = \frac{s^{input}_{n,i,t}}{R^{up}_{n,i,t}} + \max\left\{\frac{d^i_{n,t} s^{cycle}_{n,i,t}}{f^i_{n,t} F^e}, \left(\frac{(1 - d^i_{n,t}) s^{input}_{n,i,t}}{R^c} + \frac{(1 - d^i_{n,t}) s^{cycle}_{n,i,t}}{F^c}\right)\right\}, \tag{4}$$

where F^e and F^c represent the computing capacities of the ES and the CS, respectively. Otherwise, the task of TD n can only be processed at

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:53:18 UTC from IEEE Xplore. Restrictions apply.

the CS, and the corresponding delay is written as

$$T_{n,i,t}^{cloud} = \frac{s_{n,i,t}^{input}}{R_{n,i,t}^{up}} + \frac{s_{n,i,t}^{input}}{R^{c}} + \frac{s_{n,i,t}^{cycle}}{F^{c}}. \tag{5}$$

Hence, the delay for processing the task of the nth TD at the tth small-timescale slot within the ith large-timescale slot for any case can be expressed as

$$T_{n,i,t} = c_f^i T_{n,i,t}^{edge} + (1 - c_f^i) T_{n,i,t}^{cloud}. \tag{6}$$

E. Problem Formulation

In this work, we aim to design a two-timescale joint service caching and resource allocation scheme to minimize the long-term task processing delay and the cache cost, which involves both the long-term service caching scheduling and the short-term task offloading and allocation of computing and communication resource ($\mathbb{X} = \{\mathbf{c}(i), \mathbf{d}(t), \mathbf{f}(t), \mathbf{b}(t)\}$). In doing so, the optimization problem can be formulated as

$$\min_{\mathbb{X}} \lim_{I\to\infty} \frac{1}{I} \sum_{i\in\mathcal{I}} \alpha \sum_{t\in\mathcal{T}} \sum_{n\in\mathcal{N}} T_{n,i,t} + (1-\alpha) Cost(i) \tag{7}$$

$$\text{s.t.} \quad d_{n,t}^i \le \lceil f_{n,t}^i \rceil \le c_f^i, \quad \forall n \in \mathcal{N}, \forall i \in \mathcal{I}, \forall t \in \mathcal{T} \tag{7a}$$

$$\sum_{f\in\mathcal{F}} c_f^i s_f^{cache} \le C, \quad \forall i \in \mathcal{I} \tag{7b}$$

$$\sum_{n\in\mathcal{N}} f_{n,t}^i \le 1, \quad \forall i \in \mathcal{I}, \forall t \in \mathcal{T} \tag{7c}$$

$$\sum_{n\in\mathcal{N}} b_{n,t}^i \le 1, \quad \forall i \in \mathcal{I}, \forall t \in \mathcal{T} \tag{7d}$$

$$c_f^i \in \{0,1\}, \quad \forall f \in \mathcal{F}, \forall i \in \mathcal{I} \tag{7e}$$

$$d_{n,t}^i \in [0,1], \quad \forall n \in \mathcal{N}, \forall i \in \mathcal{I}, \forall t \in \mathcal{T} \tag{7f}$$

$$f_{n,t}^i \in [0,1], \quad \forall n \in \mathcal{N}, \forall i \in \mathcal{I}, \forall t \in \mathcal{T} \tag{7g}$$

$$b_{n,t}^i \in [0,1], \quad \forall n \in \mathcal{N}, \forall i \in \mathcal{I}, \forall t \in \mathcal{T} \tag{7h}$$

where $\alpha \in [0,1]$ is a preference weight factor. The constraint (7a) indicates that the task cannot be processed if there is no relevant service model cached at the ES or no computing resources have been allocated to it. Constraints (7b), (7c), (7d) define the limitations of the caching, computing, and bandwidth capacities of the ES. The constraints (7e), (7f), (7g), (7h) make (7) a two-timescale mixed-integer nonlinear programming (MINP) problem, which is difficult to solve utilizing the traditional methods. Fortunately, it is a long-term sequential decision problem that can be translated into MDP and solved by a deep reinforcement learning (DRL) algorithm. However, due to the fact that the action space includes decisions at different timescales, traditional DRL algorithms, such as DDPG, cannot be directly applied. Therefore, we propose an improved DDPG algorithm for solving this problem. The details of the algorithm are shown in the next section.

III. DRL-Based Two-Timescale Multi-Dimensional Resource Allocation Algorithm

A. Problem Transformation Based on MDP

The dynamic environment and original optimization problem can be transformed into a MDP $(\mathcal{S}; \mathcal{A}; \mathcal{R}; \mathcal{P}_{s\acute{s}})$. We treat the ES as a centralized agent and the detailed definition of state, action, reward and state transition are as follows.

State: The state space consists of five parts: a set of channel information, two sets of task information sent from TDs, two sets of service caching information calculated by the ES.

$$\mathbf{s}(t) = \{\mathbb{H}(t), \mathbb{T}_{req}(t), \mathbb{T}_{off}(t), \mathbb{C}(t), \mathbb{C}_{gain}(t)\} \tag{8}$$

where $\mathbb{H}(t) = \{h_{n,i,t}\}_{n\in N}$ represents the wireless channel gain of each TD, $\mathbb{T}_{req}(t) = \{s_{n,i,t}^{type}\}_{n\in N}$ represents the current tasks request of all TDs, $\mathbb{T}_{off}(t) = \{p_{f,i,t}\}_{f\in F}$ represents the popularity of each task, $p_{f,i,t} = \sum_{\tau=1}^{i-1} \sum_{t=1}^{T} \sum_{n=1}^{N} \mathbb{I}[s_{n,\tau,t}^{type} = f] d_{n,t}^{\tau}$ indicates the total amount of tasks of type f that have been offloaded to the ES from the beginning to the tth small-timescale slot within the ith large-timescale slot. $\mathbb{C}(t) = \{c_f^i\}_{f\in F}$ represents the current configuration of service caching, specifying which tasks the ES can process. $\mathbb{C}_{gain}(t) = \{g_{f,i,t}\}_{f\in F}$ describes the cumulative cache gain of each service model. $g_{f,i,t} = \sum_{\tau=1}^{i-1} \sum_{t=1}^{T} \sum_{n=1}^{N} \mathbb{I}[s_{n,\tau,t}^{type} = f](T_{n,\tau,t}^{edge} - T_{n,\tau,t}^{cloud})$ represents the delay gain caused by edge computing compared to cloud computing from the start to the tth small-timescale slot within the ith large-timescale slot.

Action: The action space consists of the long-term service caching decisions and the short-term task offloading, computing, and bandwidth resource allocation decisions $\mathbf{a} = \{\mathbf{c}(i), \mathbf{d}(t), \mathbf{f}(t), \mathbf{b}(t)\}$. We assume that the initial actions of the agent generated are $\tilde{\mathbf{a}} = \{\tilde{\mathbf{c}}(i), \tilde{\mathbf{d}}(t), \tilde{\mathbf{f}}(t), \tilde{\mathbf{b}}(t)\}$, whose elements are all in the range of [0, 1], and an action amender is designed to ensures that all actions, $\mathbf{a} = \{\mathbf{c}(i), \mathbf{d}(t), \mathbf{f}(t), \mathbf{b}(t)\}$, satisfy the constraints in problem (7).

- To satisfy constraint (7c), the caching indicator is quantized to a binary bit. If $\tilde{c}_f^i \in [0, 0.5]$, the final action c_f^i is 0, otherwise 1.
- To satisfy constraints (7d), (7h), the bandwidth allocation indicator is modified to be $b_{n,t}^i = \frac{\tilde{b_{n,t}^i}}{\sum_{n=1}^{N} \tilde{b_{n,t}^i}}$.
- To satisfy the constraints (7a), (7g), the computing resource allocation factor is modified to be $f_{n,t}^i = \frac{\tilde{f_{n,t}^i}}{\sum_{n=1}^{N} \mathbb{I}[c_f^i = 1] \tilde{f_{n,t}^i}}$.
- To satisfy the constraints (7a), (7f), the offloading indicator is modified to be $d_{n,t}^i = \mathbb{I}[c_f^i = 1 \& f_{n,t}^i > 0] \tilde{d_{n,t}^i}$.

Reward: To adapt to the goal of maximizing the agent's reward function, the instantaneous reward $r(t)$ contains the the negative value of the target function and the penalty,

$$r(t) = -\left(\alpha \sum_{n=1}^{N} T_{n,i,t} + (1-\alpha) Cost(i)\right) - p(t), \tag{9}$$

where $p(t) = \xi \min\{(\sum_{f=1}^{F} c_f^i s_f^{cache} - C), 0\}$ is the penalty for violating the cache space constraint (7b), ξ is a factor that balances the target function with the penalty.

State Transition: $\mathcal{P}_{s\acute{s}}$ refers to the probability that the current state, $\mathbf{s}$, moves to the next state, $\acute{\mathbf{s}}$. It should be noted that the state space consists of stochastic environmental information, such as $\mathbb{H}(t)$ and $\mathbb{T}_{req}(t)$, and their state transitions are independent of actions. On the other hand, $\mathbb{T}_{off}(t)$, $\mathbb{C}(t)$, and $\mathbb{C}_{gain}(t)$ rely entirely on actions to perform state transitions, when the actions are completed, the new states are obtained. Therefore, it is difficult for us to obtain the state transition probability of the entire system. As an alternative, we use the interactions and feedback between the agent and the environment to learn out the state transitions.

B. Centralized Dual-Actor DDPG-Based Algorithm

By regarding the formulated problem as a MDP, we use DDPG to develop effective solutions, since DDPG has an efficient and stable actor-critic training framework. But, the action space includes two

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:53:18 UTC from IEEE Xplore. Restrictions apply.

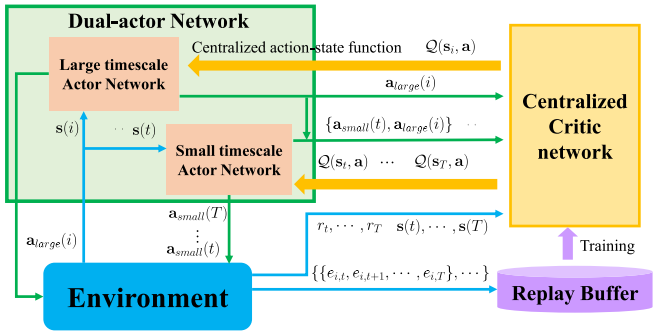


Fig. 3. Architecture of centralized dual-actor DDPG algorithm.

different timescales, the existing DDPG algorithms cannot be directly applied. To solve, we propose an improved DDPG structure, Centralized Dual-actor DDPG. The centralized Dual-actor DDPG consists of four main components: the environment, a dual-actor network, a centralized critic network, and an experience replay buffer as shown in Fig. 3.

1) Dual-Actor Network: In the dual-actor network, one actor μ_{la} is used to output the service caching strategy on a large timescale $\mathbf{a}_{large}(i) = \{\mathbf{c}(i)\}$, while the other actor μ_{sa} outputs the small-timescale decisions $\mathbf{a}_{small}(t) = \{\mathbf{d}(t), \mathbf{f}(t), \mathbf{b}(t)\}$.

$$\mathbf{a}_{large}(i) = \mu_{la}(\mathbf{s}_i|\boldsymbol{\theta}^{la}) + \eta^l, \tag{10}$$

$$\mathbf{a}_{small}(t) = \mu_{sa}(\mathbf{s}_t|\boldsymbol{\theta}^{sa}) + \eta^s, \tag{11}$$

where $s_i = s_{t=1}, \boldsymbol{\theta}^{la}, \boldsymbol{\theta}^{sa}$ are network parameters of the two actors, and η^l, η^s are random Gaussian variables, which decrease as the training progresses, and are used to encourage the dual-actor network to explore potentially better strategies.

2) Centralized Critic Network: The main function of the centralized critic network is to evaluate the joint actions $\mathbf{a} = \{\mathbf{a}_{small}(t), \mathbf{a}_{large}(i)\}$, and provide global feedback to the dual-actor network to guide decision-making. This is achieved by learning a centralized state-action value function $\mathcal{Q}(\mathbf{s}_t, \mathbf{a}|\boldsymbol{\theta}^c)$. Specifically, both μ_{sa} and μ_{la} are guided by a real-time updated centralized Q-function $\mathcal{Q}$. As shown in Fig. 3, μ_{sa} receives a real-time feedback while μ_{la} receives a intermittent feedback. And the centralized Q-function is

$$\mathcal{Q}(\mathbf{s}_t, \mathbf{a}|\boldsymbol{\theta}^c) = E\{r_t + \gamma E\{\mathcal{Q}(\mathbf{s'}_t, \mathbf{a'}|\boldsymbol{\theta}^c)\}, \tag{12}$$

where $\mathbf{s'}_t, \mathbf{a'}$ represent the next state and the joint action, respectively, $\boldsymbol{\theta}^c$ represents the critic network parameters, γ represents the discount factor utilized to indicate the degree that the current actions affect future actions.

3) Experience Pool: To tailor it to the centralized training mode of the centralized critic network, the traditional experience pool undergoes modification and sequentially stores the joint action experiences $e_{i,t} = \{\mathbf{s}_t, \{\mathbf{a}_{small}(t), \mathbf{a}_{large}(i)\}, r_t, \mathbf{s}_{t+1}\}$.

Centralized Dual-actor DDPG utilizes the experience replay technique and the two deep neural network technology to train their networks. Its core idea is to randomly sample a small batch of experiences, and use them to alternately update the parameters of the target network and the current network. The dual-actor network applies policy gradient to update their current network parameters $\boldsymbol{\theta}^{la}, \boldsymbol{\theta}^{sa}$,

$$\nabla_{\boldsymbol{\theta}^{la}} J(\boldsymbol{\theta}^{la}) \approx E[\nabla_{\boldsymbol{\theta}^{la}} \mathcal{Q}(\mathbf{s}_i, \mathbf{a}|\boldsymbol{\theta}^c) \nabla_{\boldsymbol{\theta}^{la}} \mu(\mathbf{s}_i, \boldsymbol{\theta}^{la})], \tag{13}$$

$$\nabla_{\boldsymbol{\theta}^{sa}} J(\boldsymbol{\theta}^{sa}) \approx E[\nabla_{\boldsymbol{\theta}^{sa}} \mathcal{Q}(\mathbf{s}_t, \mathbf{a}|\boldsymbol{\theta}^c) \nabla_{\boldsymbol{\theta}^{sa}} \mu(\mathbf{s}_t, \boldsymbol{\theta}^{sa})], \tag{14}$$

TABLE I
SIMULATION PARAMETERS

Number of TDs N	10
Caching interval T	5
Capabilities of ES W,C,F^e	20MHz,3000MB,20GHz
Computing capacity of CS F^c	5GHZ
Backhual transmission rate R^{back},R^c	200Mbps,100Mbps
Factors α,η, ξ	$\frac{1}{2}$, $\frac{1}{200}$,$\frac{1}{400}$
Random Gaussian variable η^l	0.9885
Random Gaussian variable η^s	0.99995
Soft update factor ζ	0.05
Large Actor network learning rate l_{la}	0.001
Small Actor network learning rate l_{la}	0.001
Critic network learning rate l_c	0.002
Discount factor γ	0.95
Memory pool capacity D	10000
Mini-batch size K	64

and the centralized critic network adjusts parameters $\boldsymbol{\theta}^c$ by minimizing the loss function

$$Loss(\boldsymbol{\theta}^c) = E[(r_t + \gamma \acute{\mathcal{Q}}(\mathbf{s'}_t, \mathbf{a'}|\acute{\boldsymbol{\theta}^c}) - \mathcal{Q}(\mathbf{s}_t, \mathbf{a}|\boldsymbol{\theta}^c))^2], \tag{15}$$

where $\mathbf{a}' = \{\acute{\mu}(\mathbf{s}_t|\acute{\boldsymbol{\theta}^{sa}}), \acute{\mu}(\mathbf{s}_i|\acute{\boldsymbol{\theta}^{la}})$ is the next joint actions output by the target dual-actor network, $\acute{\mathcal{Q}}(\mathbf{s'}_t, \acute{\mu}(\mathbf{s'}_t|\acute{\boldsymbol{\theta}^a})$ is the evaluation of the target centralized critic network for the next state and the joint actions. The parameters of target networks $\acute{\boldsymbol{\theta}^{la}}, \acute{\boldsymbol{\theta}^{sa}}, \acute{\boldsymbol{\theta}^c}$ can be softly updated according to the real-time updated parameters $\boldsymbol{\theta}^{la}, \boldsymbol{\theta}^{sa}$ and $\boldsymbol{\theta}^c$, respectively.

$$\boldsymbol{\theta}^{la'} \leftarrow \zeta \boldsymbol{\theta}^{la} + (1-\zeta)\boldsymbol{\theta}^{la'}, \tag{16}$$

$$\boldsymbol{\theta}^{sa'} \leftarrow \zeta \boldsymbol{\theta}^{sa} + (1-\zeta)\boldsymbol{\theta}^{sa'}, \tag{17}$$

$$\boldsymbol{\theta}^{c'} \leftarrow \zeta \boldsymbol{\theta}^{c} + (1-\zeta)\boldsymbol{\theta}^{c'}, \tag{18}$$

where ζ is the soft update factor.

IV. SIMULATION RESULTS AND ANALYSIS

We consider a network where there is a CS, an ES, and $[10, 20]$ TDs in a 250 m $\times$ 250 m square area. We assume the number of types of tasks and the corresponding service models $F = 10$, and the storage size of each service model $s_f^{cache} \in [20, 400]$MB. It is assumed that each TD randomly generates a task in each small-timescale slot, with the input data size $s_{n,i,t}^{input} \in [50, 500]$KB, and requires $s_{n,i,t}^{comp} \in [2 \times 10^6, 4 \times 10^9]$ number of CPU cycles to be processed. The transmit power of TDs and the ES are set to 20 dBm and 1 W, respectively. All algorithms are implemented in Python using the torch framework. Each network has three hidden layers and combines LReLU and Tanh activation functions. The Adam optimizer is used. The parameters of the training stage for the algorithm are presented in Table I.

To evaluate the performance of the Centralized Dual-actor DDPG (**Proposed-DDPG**), the default parameters deployed in the simulations are summarized in Table I, and the following benchmarks are adopted. They are:

- **Independent-DDPG algorithm**: two independent DDPG (actor-critic) networks are used to get the service caching and the short-term resource allocation strategies.
- **Average scheme**: evenly allocates the bandwidth and computing resources to TDs, whose tasks are unloaded equally between the

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:53:18 UTC from IEEE Xplore. Restrictions apply.

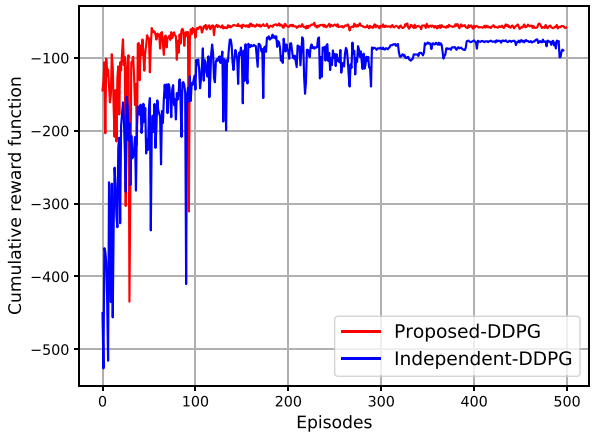


Fig. 4. Convergence performance of accumulative reward for Proposed-DDPG.

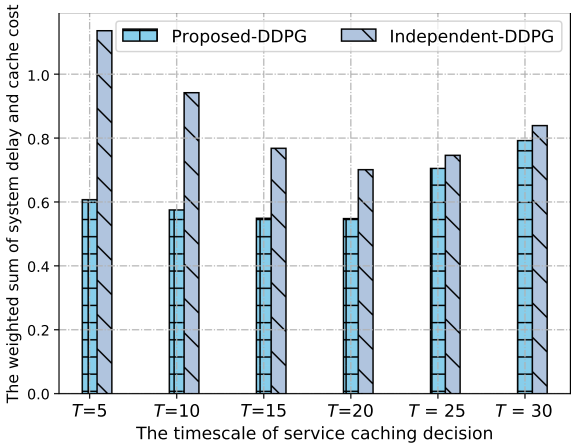


Fig. 5. Influence of caching timescale T.

ES and the CS, while the service caching is optimized based on the proposed scheme.

- **Random-cache scheme**: caches services randomly and other decisions are based on the proposed scheme.

1) The Convergence Performance of Training Phase: The cumulative rewards of the Proposed-DDPG and Independent-DDPG in the training phase for $T = 5$ are shown in Fig. 4. It is seen that the Proposed-DDPG converges faster and is more stable than Independent-DDPG, and there are strong oscillations in Independent-DDPG which are caused by the asynchronous evaluation of the two independent actor-critic networks. This verifies the importance of the proposed centralized critic network, which guides the decision of the dual-actor network from a global long-term perspective.

2) The Utility Versus Different Timescale T: The impact of the timescale T, i.e., the frequency for updating the service caching, on the utility function is also investigated. Comparisons are made between the Proposed-DDPG and the Independent-DDPG, and the results are shown in Fig. 5. It is seen that as the caching interval, T, increases, the utility first decreases and then increases for both the two approaches, and the Proposed-DDPG outperforms the Independent-DDPG for all the simulated timescales. The simulation results reveal that the caching decision has a significant impact on offloading and resource allocation, when the caching interval is small. Since the centralized critic network provides more accurate evaluation than two independent actor-critic networks, the gain brought by the Proposed-DDPG is more significant. When the caching interval increases to $T > 20$, the impact of caching decreases, but the proposed algorithm still outperforms the Independent-DDPG, verifying the effectiveness of the proposed algorithm in solving multi-dimensional resource allocation problems in two-timescale.

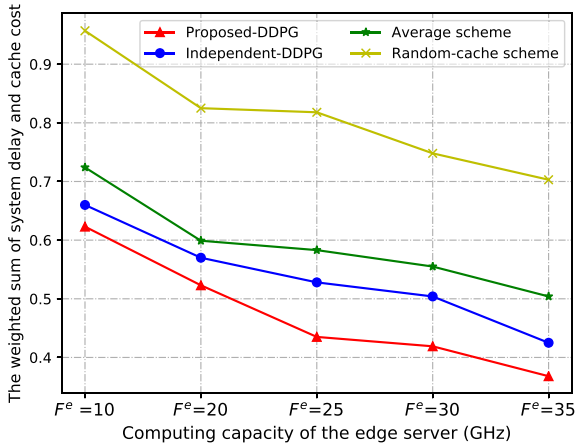


Fig. 6. Influence of the computing capacity of the ES.

3) The Utility Versus Computing Ability F^e: The impact of the ES computing resources on the weighted sum of delay and cache cost are simulated when $N = 10, T = 5$, and the results are shown in Fig. 6. It is seen that the utility improves as the computing resources increases, and the Proposed-DDPG consistently outperforms all the baselines obviously. This further highlights the advantages of jointly optimizing two-timescale service caching, task offloading and resource allocation.

V. Conclusion

In this paper, we jointly considered the edge service caching, task offloading, and resource allocation processes in MEC networks with different timescales, and formulated a two-timescale multi-dimensional resource joint optimization problem to minimize the weighted long-term system task processing delay and the cache cost. Due to the strong coupling relationship among multi-dimensional resources, it is difficult to achieve long-term optimum solutions via traditional algorithms. To solve the optimization problem, we first transformed it into a MDP and then proposed a novel Centralized Dual-actor DDPG approach by establishing a centralized critic network and a dual-actor network. The dual-actor network can provide flexible strategies for the long-term service caching and the short-term task offloading, computing, and bandwidth resource allocation, while the centralized critic network can ensure the long-term consistency among all decision outputs, making it converge faster and more stable. Simulation results also confirm that the Proposed-DDPG is more effective in handling two-timescale problems compared to the Independent-DDPG.

References

[1] T. Qiu, J. Chi, X. Zhou, Z. Ning, M. Atiquzzaman, and D. O. Wu, "Edge computing in industrial Internet of Things: Architecture, advances and challenges," *IEEE Commun. Surv. Tut.*, vol. 22, no. 4, pp. 2462–2488, Fourth Quarter 2020.

[2] Y. Deng, X. Chen, G. Zhu, Y. Fang, Z. Chen, and X. Deng, "Actions at the edge: Jointly optimizing the resources in multi-access edge computing," *IEEE Wireless Commun.*, vol. 29, no. 2, pp. 192–198, Apr. 2022.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:53:18 UTC from IEEE Xplore. Restrictions apply.

[3] X. Dai et al., "Task co-offloading for D2D-assisted mobile edge computing in industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 480–490, Jan. 2023.

[4] S. Bi, L. Huang, and Y.-J. A. Zhang, "Joint optimization of service caching placement and computation offloading in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 19, no. 7, pp. 4947–4963, Jul. 2020.

[5] S.-W. Ko, S. J. Kim, H. Jung, and S. W. Choi, "Computation offloading and service caching for mobile edge computing under personalized service preference," *IEEE Trans. Wireless Commun.*, vol. 21, no. 8, pp. 6568–6583, Aug. 2022.

[6] Z. Li, C. Yang, X. Huang, W. Zeng, and S. Xie, "CoOR: Collaborative task offloading and service caching replacement for vehicular cdge computing networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 7, pp. 9676–9681, Jul. 2023.

[7] J. Yan, S. Bi, L. Duan, and Y.-J. A. Zhang, "Pricing-driven service caching and task offloading in mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 20, no. 7, pp. 4495–4512, Jul. 2021.

[8] G. Zhang, S. Zhang, W. Zhang, Z. Shen, and L. Wang, "Joint service caching, computation offloading and resource allocation in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 20, no. 8, pp. 5288–5300, Aug. 2021.

[9] W. Chu, P. Yu, Z. Yu, J. C. S. Lui, and Y. Lin, "Online optimal service selection, resource allocation and task offloading for multi-access edge computing: A utility-based approach," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 4150–4167, Jul. 2023.

[10] H. Zhou, Z. Wang, H. Zheng, S. He, and M. Dong, "Cost minimization-oriented computation offloading and service caching in mobile cloud-edge computing: An A3C-based approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 3, pp. 1326–1338, May/Jun. 2023.

[11] R. Zhou, X. Wu, H. Tan, and R. Zhang, "Two time-scale joint service caching and task offloading for UAV-assisted mobile edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1189–1198.

[12] W. Wen, Y. Cui, T. Q. S. Quek, F.-C. Zheng, and S. Jin, "Joint optimal software caching, computation offloading and communications resource allocation for mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7879–7894, Jul. 2020.

[13] J. Zhang, Y. Shen, Y. Wang, X. Zhang, and J. Wang, "Dual-timescale resource allocation for collaborative service caching and computation offloading in IoT systems," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1735–1746, Feb. 2023.

[14] T. Xue, H. Ding, H. Zhang, and D. Yuan, "Two-timescale vehicle association and resource management for C-V2X networks," *IEEE Wireless Commun. Lett.*, vol. 12, no. 7, pp. 1259–1263, Jul. 2023.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:53:18 UTC from IEEE Xplore. Restrictions apply.